



1.

PROJECT REPORT

PASSWORD STRENGTH CHECKER

NAME :

REG.NO.:

COURSE:

2. INTRODUCTION

- PASSWORDS ARE THE FIRST LINE OF DEFENCE FOR OUR ONLINE ACCOUNTS
- MANY PEOPLE USE WEAK PASSWORDS OR THE SAME PASSWORD EVERYWHERE.
- THIS PROJECT IS A PYTHON TOOL THAT CHECKS PASSWORD STRENGTH AND GIVES TIPS.
- IT ALSO FINDS REUSED PASSWORDS AND SUGGESTS STRONG NEW ONES.

Passwords are our first line of defense ,yet commom user habits create massive vulnerability

A Moduler python utility designed to evaluate strength, detect cross app reuse and securely generate passwords

Built to bridge the gap between complex cryptographic safety and everyday user convenience



3.

PROBLEM STATEMENT :

- PEOPLE USE EASY PASSWORDS LIKE 'PASSWORD123' OR 'QWERTY'.
- ONE PASSWORD IS OFTEN USED FOR MANY APPS, SO ONE HACK CAN OPEN ALL ACCOUNTS.
- MOST USERS DO NOT KNOW IF THEIR PASSWORD IS STRONG OR HOW TO IMPROVE IT.

GOAL:

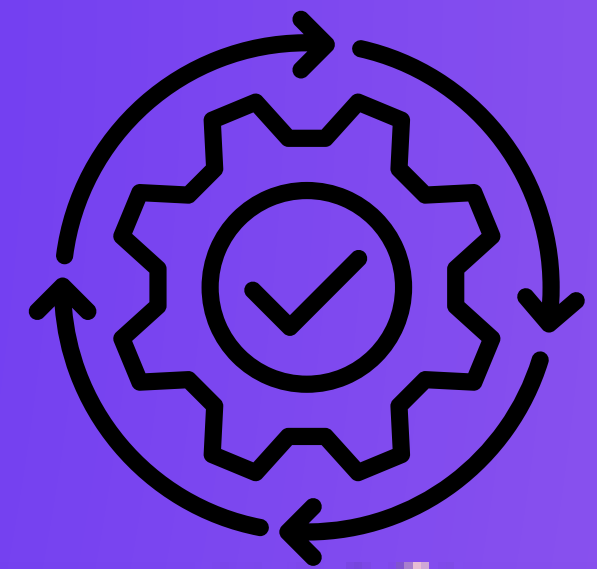
- A SIMPLE TOOL THAT CHECKS, EXPLAINS AND SUGGESTS BETTER PASSWORDS.
- "I CHOSE THIS PROJECT BECAUSE CREDENTIAL STUFFING AND BRUTE-FORCE ATTACKS REMAIN THE TOP METHODS HACKERS USE TO BREACH INDIVIDUAL AND ENTERPRISE NETWORKS".



4. FUNCTIONAL REQUIREMENTS

- STRENGTH CHECKER: SCORE OUT OF 6 AND RATING (WEAK TO VERY STRONG).
- REUSE DETECTOR: WARNS IF ONE PASSWORD IS USED IN MANY APPS.
- PASSWORD GENERATOR: MAKES RANDOM STRONG PASSWORDS. INPUT: APP NAME AND PASSWORD.
- OUTPUT: SCORE, RATING, TIPS, WARNINGS.
- WORKFLOW:

ENTER APP + PASSWORD > CHECK > SHOW RESULT > REPEAT > REUSE CHECK.



5. NON- FUNCTIONAL REQUIREMENTS



SECURITY: USES
THE 'SECRETS'
MODULE;
PASSWORDS ARE
NEVER SAVED OR
LOGGED.



Usability:
simple
messages
and clear
tips in the
terminal.



*Error
handling:*
empty input
and wrong
input are
checked.



Maintainability:
code is split
into small files
with
comments.



PERFORMANCE:
RESULT APPEARS
INSTANTLY FOR
NORMAL
PASSWORDS.

6. SYSTEM ARCHITECTURE

A close-up, slightly blurred image of a hand with fingers pressing keys on a computer keyboard. The lighting is dim, with some highlights on the keys and the hand.

USER =>

MAIN.PY(INPUT/OUTPUT), VALIDATOR.PY

MAIN.PY , CHECKER.PY , RATING.PY

MAIN.PY , REUSE_CHECKER.PY , GENERATOR.PY

LOGGER.PY=> WRITES ACTIVITY TO A LOG FILE

COMMON_PASSWORDS.TXT=> IS READ BY

CHECKER.PY

7. DESIGN OVERVIEW

PASSWORD STRENGTH CHECKER — Design Overview

Project Overview

Evaluates password strength, provides actionable security tips, and warns the user about password reuse across saved accounts.

Core Functions

- **check_password()**: Strength evaluation
- **get_rating()**: Score conversion to WEAK/MEDIUM/STRONG
- **make_strong_password()**: Secure password generation

Key Features

- ✓ Score 0–6
- ✓ Common password detection
- ✓ Character diversity analysis
- ✓ Pattern detection
- ✓ Reuse detection across apps
- ✓ Secure password generation

Validation Criteria

- **Length**: 2 pts (≥12 chars) / 1 pt (≥8 chars)
- **Uppercase**: 1 point
- **Lowercase**: 1 point
- **Digits**: 1 point
- **Special chars**: 1 point
- **Repeated patterns**: -1 point penalty



Use Case Workflow



Sequence Components

User → main → check / rating → return

check()

Runs the strength evaluation on the entered password

rating()

Converts the numeric score into a readable rating

generate()

Produces a new cryptographically secure password

Main

Coordinates calls and returns results to the user



9. IMPLEMENTATION DETAILS



10. SCREENSHOTS / RESULTS

(01)

Problems Output Debug Console **Terminal** Ports

```
App name (or 'done'): instagram  
Password for instagram: samya123
```

```
Strength: WEAK (2/6)
```

```
How to improve:
```

- Make it at least 12 characters long.
- Add at least one UPPERCASE letter.
- Add at least one special character like ! @ # \$ %
- Avoid easy patterns like '123' or 'abc'.

1. WEAK PASSWORD (SAMYA123) → WEAK 2/6 WITH 4 TIPS

02

```
Problems Output Debug Console Terminal Ports

App name (or 'done'): snapchat
Password for snapchat: Xk9!mQ2p@Lw7T

Strength: VERY STRONG (6/6)
Great password! Nothing to improve.
```

2. STRONG PASSWORD → VERY STRONG 6/6, "NOTHING TO IMPROVE"

03



3. COMMON PASSWORD → WEAK 0/6, CAUGHT IMMEDIATELY

04

App name (or "done"): facebook
Password for facebook: F23m@n

Strength: MEDIUM (4/6)

How to improve:

- Too short! Use at least 8 characters (12 or more is better).

App name (or "done"): X

Password for X: F23m@n

Strength: MEDIUM (4/6)

How to improve:

- Too short! Use at least 8 characters (12 or more is better).

App name (or "done"): done

=== Reuse Check ===

Same password used in: facebook, X

If one app gets hacked, hackers can enter all of these!

Use a different password for each app. Suggestions:

facebook -> y%lp5K@5cdw!\$"

X -> 8X3n@bd2up1ic

Stay safe online!

4. REUSE CHECK → SAME PASSWORD ON FACEBOOK & X, WARNING + SUGGESTIONS

```
Problems Output Debug Console Terminal Ports
test_score_never_goes_below_zero (test_password_checker.TestCheckPassword.test_score_never_goes_below_zero) ... ok
test_short_password_gets_low_score (test_password_checker.TestCheckPassword.test_short_password_gets_low_score) ... ok
test_strong_password_gets_high_score (test_password_checker.TestCheckPassword.test_strong_password_gets_high_score) ... ok
test_five_is_strong (test_password_checker.TestGetRating.test_five_is_strong) ... ok
test_four_is_medium (test_password_checker.TestGetRating.test_four_is_medium) ... ok
test_six_is_very_strong (test_password_checker.TestGetRating.test_six_is_very_strong) ... ok
test_three_is_medium (test_password_checker.TestGetRating.test_three_is_medium) ... ok
test_two_is_weak (test_password_checker.TestGetRating.test_two_is_weak) ... ok
test_zero_is_weak (test_password_checker.TestGetRating.test_zero_is_weak) ... ok
test_custom_length_is_respected (test_password_checker.TestMakeStrongPassword.test_custom_length_is_respected) ... ok
test_default_length_is_14 (test_password_checker.TestMakeStrongPassword.test_default_length_is_14) ... ok
test_generated_password_scores_well (test_password_checker.TestMakeStrongPassword.test_generated_password_scores_well) ... ok
test_two_generated_passwords_are_different (test_password_checker.TestMakeStrongPassword.test_two_generated_passwords_are_different) ... ok

.....
Ran 18 tests in 0.003s

OK
```

5. TEST RUN → 18 TESTS, ALL OK, FINAL OK

11. TESTING APPROACH



- COMMON PASSWORD LIKE 'PASSWORD' SHOULD GIVE SCORE 0.
- STRONG PASSWORD SHOULD GIVE A HIGH SCORE AND NO TIPS.



SAME PASSWORD IN 2 APPS SHOULD BE FOUND BY THE REUSE CHECK



- GENERATED PASSWORD SHOULD HAVE THE CORRECT LENGTH.
- ALSO TRY EMPTY INPUT AND VERY LONG PASSWORDS BY HAND.

12. CHALLENGES FACED

- DECIDING HOW MANY POINTS EACH RULE SHOULD GIVE.
- HANDLING REPEATED LETTERS AND EASY PATTERNS LIKE 'ABC' OR '123'.
- SPLITTING ONE BIG FILE INTO MANY MODULES.
- WRITING TESTS FOR A PROGRAM THAT USES INPUT().



the challenge

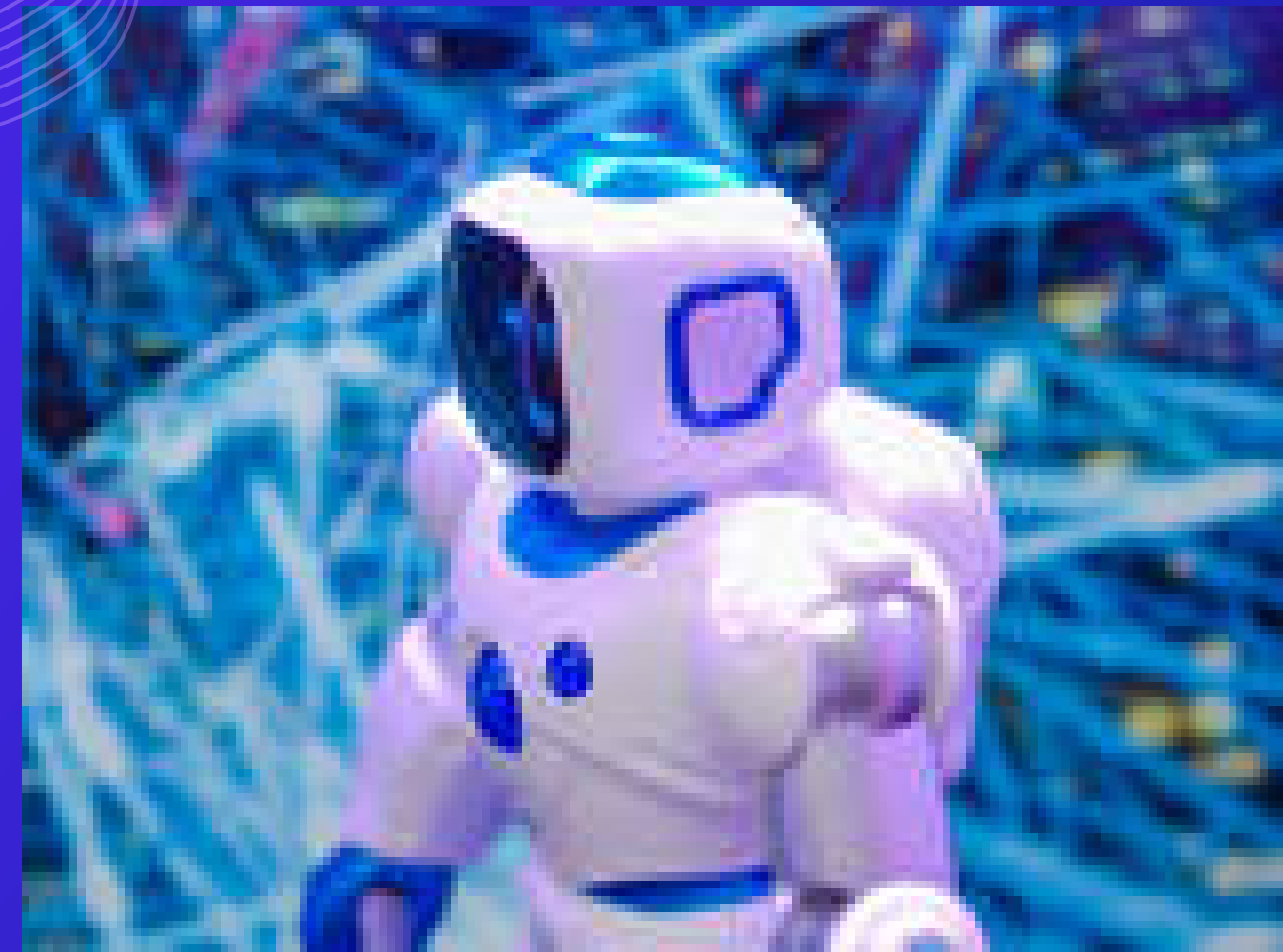
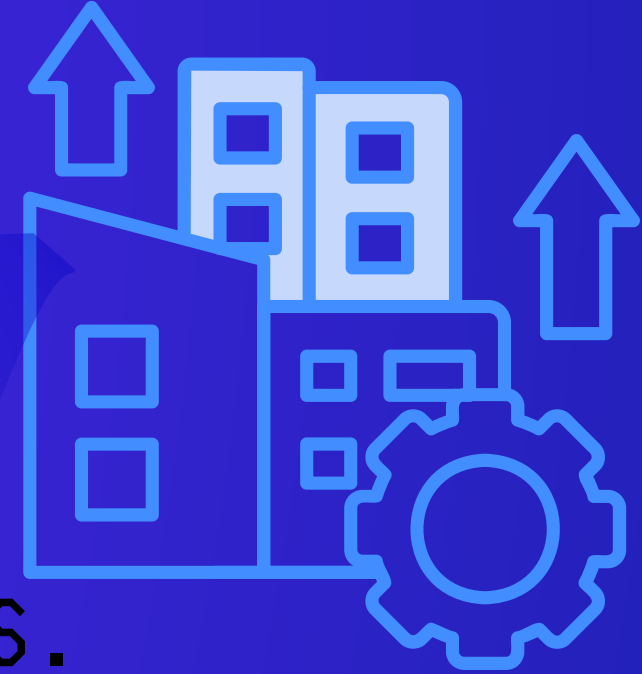
13. LEARNINGS & KEY TAKEAWAYS

- WHY STRONG AND UNIQUE PASSWORDS MATTER.
- HOW TO USE THE SECRETS MODULE INSTEAD OF RANDOM FOR SECURITY.
- HOW TO PLAN, SPLIT AND TEST A PROJECT IN MODULES.
- HOW TO USE GIT AND GITHUB FOR VERSION CONTROL.



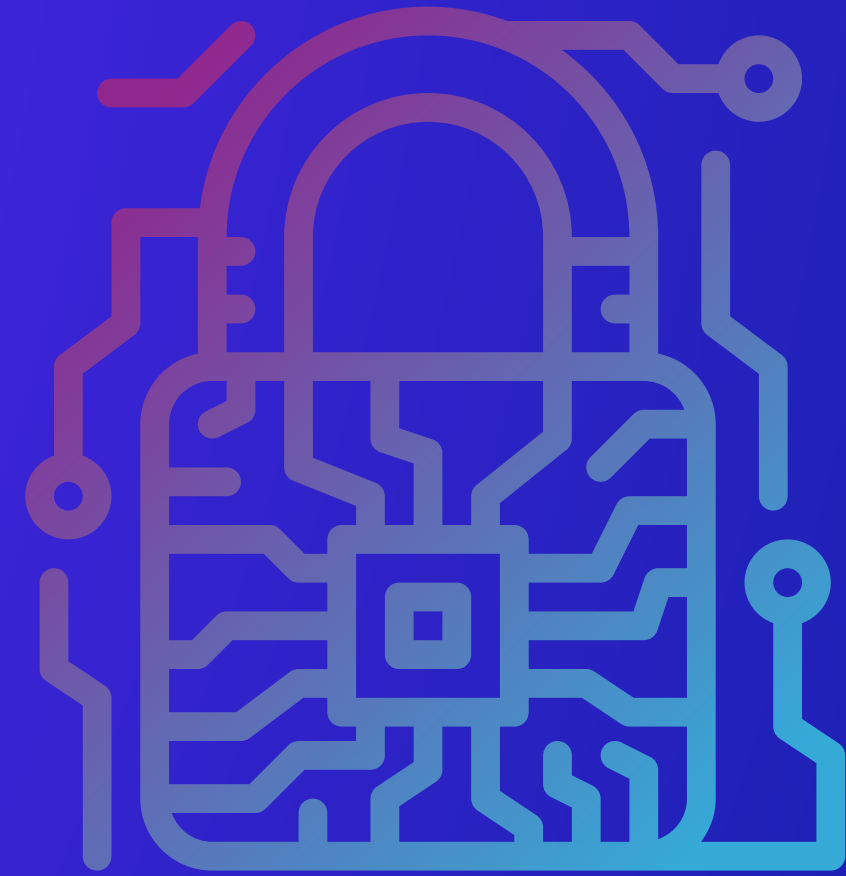
14. FUTURE ENHANCEMENTS

- ADD A SIMPLE GUI.
- CHECK PASSWORDS AGAINST A BIGGER LIST OF LEAKED PASSWORDS.
- SAVE A REPORT OF RESULTS WITHOUT SAVING THE REAL PASSWORDS.
- ESTIMATE HOW LONG A PASSWORD WOULD TAKE TO CRACK.
- MAKE A WEB VERSION.



15. REFERENCES

- PYTHON DOCUMENTATION: [DOCS.PYTHON.ORG](https://docs.python.org) (SECRETS, STRING, UNITTEST)
- OWASP AUTHENTICATION CHEAT SHEET: [OWASP.ORG](https://owasp.org)
- NIST PASSWORD GUIDELINES (SP 800-63B)



THANK YOU!

